

Airline Review RAG system

Chat agent that answers user questions from airline reviews

Bishesh Khadgi
2-3-2026

Table of Contents

List of Figures	2
Glossary of Terms	3
1. Introduction	4
1.1 Purpose	4
1.2 Overview	4
2. System Architecture	5
2.1 Architectural Design	5
2.2 Decomposition Description	6
2.3 Design Rationale	6
3. Engine	7
3.1 Data Flow Diagram	7
3.2 Breakdown	8
3.2.1 Data Source	8
3.2.2 Creating Chunks	8
3.2.3 Creating Embeddings from Chunks	8
3.2.4 Create vectorDB and store embeddings	8
3.2.5 User query	8
3.2.6 User query vector embedding	8
3.2.7 Search function	8
3.2.8 Top-k chunks	9
3.2.9 Augment prompts	9
3.2.10 LLM and response	9

List of Figures

- Figure 1: Airline Reviews RAG Agent - System Architecture
- Figure 2: Data Flow Diagram - Data Ingestion
- Figure 3: Data Flow Diagram - Data Retrieval & Generation

Glossary of Terms

- **Augmentation:** The process of strategically merging a user's query with retrieved data to provide the LLM with a context-rich instruction set.
- **ChromaDB:** The specific vector database used in this project to store and index high-dimensional embeddings for fast retrieval.
- **Cosine Distance:** A mathematical metric used to measure the similarity between two vectors; values closer to 0 indicate higher similarity.
- **Embeddings:** Numerical, high-dimensional vector representations of text that capture semantic meaning.
- **HNSW (Hierarchical Navigable Small World):** A graph-based algorithm used by the vector database to perform efficient Approximate Nearest Neighbor (ANN) searches.
- **LLM (Large Language Model):** The core reasoning engine (in this case, Llama 3.2) that synthesizes the final natural language response based on provided context.
- **Metadata:** Supplemental information stored alongside vectors, such as airline names, used to provide context to the LLM during generation.
- **Prompt Template:** A structured wrapper used to organize system instructions, context, and user queries before they are sent to the LLM.
- **RAG (Retrieval-Augmented Generation):** An architectural approach that "grounds" an LLM's response in specific, retrieved data to reduce hallucinations and improve accuracy.
- **RecursiveCharacterTextSplitter:** A utility used to divide long reviews into smaller, overlapping chunks to maintain local context.
- **Squared L2 Norm:** Also known as Euclidean Distance; the default distance metric used in this project's search function to identify relevant data points.
- **Top-k:** The predefined number (e.g., 5) of the most semantically relevant document chunks selected to be sent to the LLM.
- **Vector Database:** A specialized storage layer designed to persist and efficiently query high-dimensional embeddings.

1. Introduction

1.1 Purpose

This document describes the implementation design and architectural rationale of the **Airline Reviews Retrieval-Augmented Generation (RAG) agent**. It explains why specific design choices were made across the system, including data processing, retrieval, and response generation components.

The primary purpose of this document is to enable learning and understanding RAG, while also serving as a reference for future understanding of the system design.

1.2 Overview

The system architecture section provides a high-level view of the overall design of the Airline Reviews RAG agent. The architecture was intentionally kept minimal and modular to support clarity, maintainability, and incremental development. This approach aligns with the project's design methodology, which emphasizes simplicity and avoids heavy upfront architectural complexity.

Subsequent sections of this document describe the individual components and modules that make up the system, such as data ingestion, text chunking, vector storage, retrieval, and response generation. Together, these sections serve as a centralized reference for design rationale, implementation details, and identified areas for future enhancement.

2. System Architecture

2.1 Architectural Design

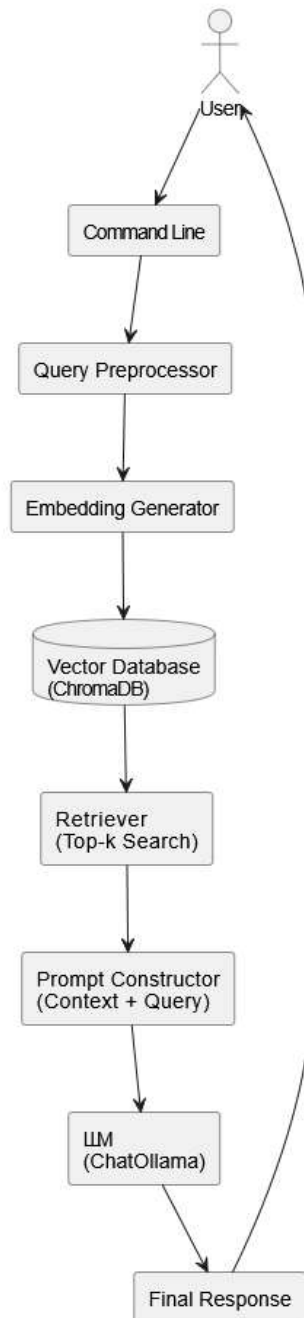


Figure 1: System Architecture

2.2 Decomposition Description

The architecture of the Airline Reviews RAG agent is composed of three primary components: the **client interface**, the **RAG processing engine**, and the **vector and metadata storage layer**. The system is decomposed in this manner to ensure separation of concerns and to allow individual components to evolve independently.

The client interface is responsible solely for accepting user queries and displaying generated responses. It does not perform any retrieval or generation logic and can operate independently of the underlying engine.

The RAG processing engine handles all computational tasks, including query preprocessing, embedding generation, vector search, context assembly, and interaction with the language model. This component encapsulates the core intelligence of the system.

The storage layer consists of a vector database that persists embedded airline review chunks along with their associated metadata. This layer enables efficient semantic retrieval and ensures that embeddings are reusable across sessions without requiring recomputation.

2.3 Design Rationale

The client interface was designed as a command line to keep the program simple. This allows the system to support multiple interfaces usage in future (e.g., web UI, API clients) without requiring changes to the core retrieval and generation logic.

The RAG processing engine follows a modular pipeline-based design. Each stage—preprocessing, embedding, retrieval, and generation—has a clearly defined responsibility. This design improves maintainability, enables easier debugging, and allows individual components to be replaced or upgraded (for example, swapping embedding models or vector stores) with minimal impact on the rest of the system as each step has a separate function handling one single task.

A vector database was selected to support semantic search over airline reviews, as traditional keyword-based search is insufficient for capturing user intent. Persisting embeddings reduces computational overhead and improves query latency. Cosine distance-based similarity search was chosen due to its effectiveness in comparing high-dimensional embedding vectors.

The use of a Retrieval-Augmented Generation approach was a deliberate design decision to reduce hallucination and improve answer reliability. By grounding responses in retrieved airline reviews, the language model is constrained to generate outputs that are directly supported by the underlying data, thereby increasing transparency and trust in the system's responses.

3. Engine

3.1 Data Flow Diagram

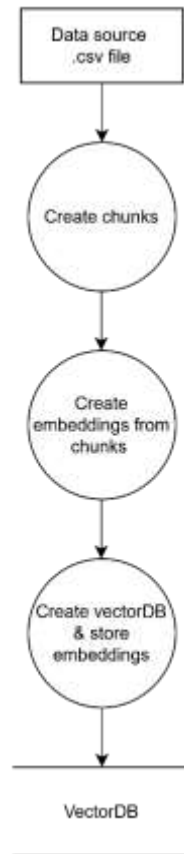


Figure 2: Data ingestion

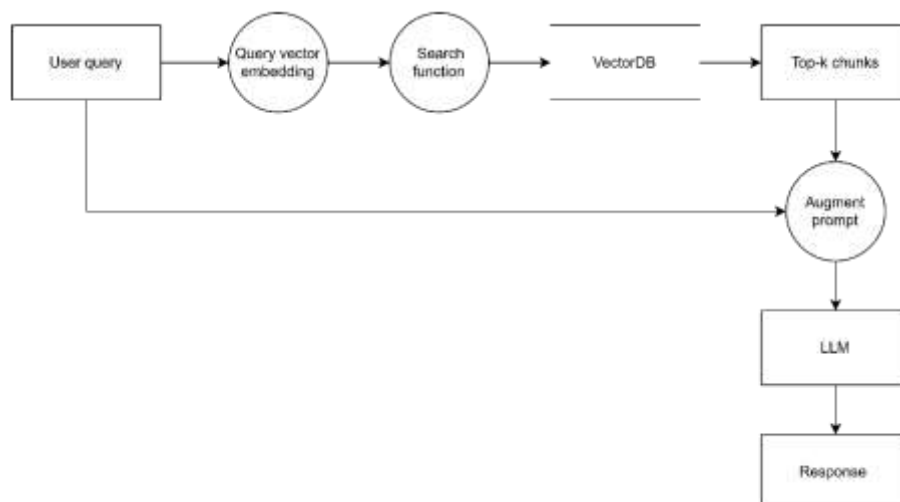


Figure 3: Data Retrieval and Generation

3.2 Breakdown

3.2.1 Data Source

The data was collected from Kaggle. It is in a comma-separated-values format containing reviews of 10 top rated airlines in 2023 sourced from Quality (<https://www.airlinequality.com>) website. The reviews cover various aspects of the flight experience, including seat comfort, staff service, food and beverages, inflight entertainment, value for money, and overall rating. The data is divided into more 15 columns like 'Title', 'Name', 'Airline', 'Review', 'Overall Rating' and so on. From all these columns, we have used review and airline column to keep the program simple and the data minimal whilst maintaining the objective of using reviews for the RAG system.

3.2.2 Creating Chunks

The reviews that were received from the csv file were then split into chunks along with the airlines of which the reviews are written for. The model used for chunking was 'RecursiveCharacterTextSplitter' which is a Langchain utility. It supports chunk overlapping which helps in keeping related texts together.

3.2.3 Creating Embeddings from Chunks

Embeddings are vector representation of the text. This step transforms textual chunks into numerical representation i.e a high dimensional vector. The ollama embedding model 'mxbai-embed-large' is used for embeddings.

3.2.4 Create vectorDB and store embeddings

This step initializes a vector database which was used to store the vector embeddings. ChromaDB is used in the project for this purpose. It can run in memory or be saved to the local directory (persistent directory) so that the data remains for later use. ChromaDB created an index and allows the system to perform searches very fast making it possible to find relevant chunks in milliseconds. Along with the embedding vector, ChromaDB stores original text and metadata, which is the airline name in the project.

3.2.5 User query

In a RAG system, user query is the starting point of retrieval process. It is the raw input provided by the user that the system needs to understand and answer. The user provides a natural language command or question on which the searching is done.

3.2.6 User query vector embedding

This step acts as the "translation layer" that converts the user's natural language question into a mathematical format, which is a vector, that the search function can compare against the stored data. We used the same model as used in chunk embeddings.

3.2.7 Search function

The search function is the main engine of the retrieval process. In ChromaDB, the search function follows a specific mathematical process to identify the most relevant data points. By default, ChromaDB utilizes HNSW (Hierarchical Navigable Small World) graphs to perform Approximate Nearest Neighbor (ANN) searches. The distance metric used is Squared L2 Norm which utilizes Euclidean Distance metric whose value ranges from 0 to 2 where closer to 0 is similar match and closer to 2 is vectors diverging.

3.2.8 Top-k chunks

Top-k refers to the specific number of most relevant document segments (chunks) that the system selects to provide as context to the LLM. It acts as a process of shortlisting ensuring only the most semantically related information is used for response generation.

3.2.9 Augment prompts

Prompt Augmentation is the strategic phase where the system merges the user's original query with the retrieved top-k chunks to create a single, context-rich instruction for the LLM. Prompt serves as a wrapper that is used to pass the user query, retrieved reviews and context along with the chat history to the LLM in the program. 'ChatPromptTemplate' and 'MessagesPlaceholder' were used in the program for prompt augmentation. These tools help in organizing the messages in a structured way using roles like 'system', 'user', 'assistant'. 'MessagesPlaceholder' acts like a dynamic slot which was used to pass in a list of messages.

3.2.10 LLM and response

This was the final phase of the RAG system where the Large Language Model (LLM) acts as a reasoning engine to synthesize the final answer. The LLM receives the structured input containing the system instructions, the retrieved context and user query. The context was stuffed into the prompt, the model uses its Attention Mechanism to prioritize the retrieved facts over its own internal weights. The model used was Ollama's 'llama3.2' model with a temperature of 0.2. The low temperature makes the output more deterministic and factual discouraging the model from being 'creative' or hallucinating information not present in the vectorDB.